SIMATS ENGINEERING

ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Cloud Computing and Big Data Analytics (CSA15)

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.

All 5 Questions – Detailed Answers | Total: 50 Marks

The question paper contains five constraint-based design problems. The answers below explicitly identify the stated constraints, propose a feasible Hadoop-oriented architecture, and justify the design choices.

1. HDFS-based storage solution for a media platform

Given constraints: The media platform requires data availability above 99.9%, the replication factor must not exceed 3, and storage growth must remain cost efficient.

Proposed design: Use HDFS as the distributed storage layer with a replication factor of **3** for critical media data. Deploy multiple DataNodes across failure domains and use NameNode high availability so that metadata management does not depend on one active NameNode.

Storage layout:
• Store large media objects as HDFS files and allow HDFS to divide them into blocks.
• Use a replication factor of 3 for important data so that a block has multiple copies.
• Place replicas across different DataNodes and, where possible, different racks/failure domains.
• Use HDFS monitoring to identify failed DataNodes and under-replicated blocks.
• Add DataNodes as the media library grows instead of continuously replacing the entire storage system.

Availability: If one DataNode fails, another replica can serve the affected block. NameNode high availability reduces the risk of metadata becoming a single point of failure. Health monitoring and automatic re-replication help restore the desired redundancy after failures.

Cost efficiency: The replication factor is limited to 3, exactly satisfying the stated upper limit without using unnecessary additional copies. Capacity can be expanded horizontally by adding storage nodes as demand grows. Large media files also make distributed block storage efficient because HDFS is intended for large-scale datasets.

Justification: This design prioritizes availability through three-way replication and redundant metadata management while controlling cost by never exceeding the replication factor of 3 and scaling capacity incrementally.

2. MapReduce log processing within a 20-minute batch window

Given constraints: The workflow must process log data within a **20-minute batch window** and the cluster has a limited number of nodes.

Proposed job design: Use a MapReduce pipeline in which the log files are divided into balanced input splits. Multiple mapper tasks process the splits in parallel, followed by shuffle/sort and a controlled number of reducers.

Map phase:
• Store the incoming log files in HDFS.
• Split the input into reasonably balanced blocks/splits.
• Each mapper parses its assigned log records.
• Emit useful intermediate key-value pairs, such as **(status_code, count)**, **(service, count)**, or **(time_window, count)**, depending on the required analysis.

Shuffle and reduce: Hadoop groups mapper output by key and sends each key to the appropriate reducer. Reducers aggregate the values and write the final results back to HDFS.

Task distribution under limited nodes:
• Use enough mapper tasks to keep all available worker resources busy without creating excessive task overhead.
• Choose input split sizes that avoid extremely small tasks and also avoid oversized tasks that become stragglers.
• Configure the reducer count according to available CPU and memory rather than simply maximizing the number of reducers.
• Use a combiner for associative aggregation operations such as counts or sums. This reduces intermediate data

transferred during shuffle.
• Monitor the slowest tasks because a single straggler can determine whether the 20-minute deadline is met.

20-minute target: First estimate input size and processing throughput, then allocate mapper/reducer parallelism within the limited cluster capacity. The design should be tested using representative batches and adjusted based on actual task execution times.

Justification: Parallel mapping, balanced partitions, local HDFS data processing, optional combiners and controlled reducer allocation reduce unnecessary network and disk work while making the best use of limited nodes.

3. YARN scheduler for ETL, reporting and machine-learning jobs

Given constraints: The YARN cluster must support ETL, reporting and machine-learning workloads. The policy must provide fairness, give ETL priority, and recover from node failure.

Proposed scheduler policy: Use YARN queues with separate resource allocations for the three workload categories. Give the ETL queue higher priority because ETL is explicitly required to receive priority, while still reserving resources for reporting and machine-learning jobs.

Queue design:
• **ETL queue:** Higher priority and a guaranteed minimum capacity so scheduled data-ingestion and transformation jobs can run promptly.
• **Reporting queue:** Reserved capacity for interactive or scheduled reports so reporting is not completely blocked by long-running workloads.
• **ML queue:** Dedicated capacity for training jobs, which can consume significant CPU and memory and may run for longer periods.

Fairness: Within each queue, use fair resource allocation so one user's applications cannot consume the entire queue. Configure maximum resource limits and application limits where necessary.

Priority: ETL receives higher scheduling priority, but priority should not mean unlimited resource consumption. Minimum and maximum capacities maintain fairness across the organization.

Node failure recovery:
• YARN detects unavailable NodeManagers.
• Failed containers can be restarted on healthy nodes.
• Applications should be designed to tolerate task/container failure.
• Important workloads should have checkpointing or restart capability where supported.
• Monitor node health and cluster capacity so failed nodes can be replaced or repaired.

Justification: Queue isolation prevents ETL, reporting and ML workloads from competing without control. Priority satisfies the ETL requirement, fair allocation protects other users, and YARN's resource management and task recovery mechanisms help maintain processing after node failures.

4. Enterprise ingestion from SAN, NAS and operational databases

Given constraints: The enterprise must ingest data from SAN, NAS and operational databases while maintaining reliable backups and minimizing duplicate records.

Proposed architecture: Use a controlled ingestion layer, such as Apache NiFi or an ETL pipeline, between the source systems and the Hadoop platform. HDFS acts as the scalable landing/storage layer.

Source ingestion:
• **SAN:** Access required files or exported datasets through controlled enterprise interfaces and ingest them into the pipeline.
• **NAS:** Monitor or read shared files through appropriate file-based ingestion mechanisms.
• **Operational databases:** Use database connectors or controlled extraction jobs to capture records, preferably using incremental extraction where supported.

Ingestion and deduplication:
• Assign a unique source record identifier or business key.
• Track file names, timestamps, source identifiers and processing state.
• Use NiFi/ETL routing and transformation to validate records before loading.
• Make writes idempotent so retrying a successful ingestion does not create another copy.
• Apply deduplication before publishing data to analytics storage.

Hadoop storage: Store raw/source data in an HDFS landing zone and transformed data in separate logical areas. This preserves the original data while allowing cleaned datasets to be generated for analytics.

Backup reliability: Maintain verified backups/snapshots and track backup versions and timestamps. Test restoration regularly so that backup availability is proven rather than assumed.

Justification: A central ingestion layer gives the organization one controlled path for heterogeneous sources. Unique identifiers and state tracking reduce duplication, while HDFS provides scalable distributed storage and backup/version

controls improve recovery reliability.

5. End-to-end Hadoop ecosystem solution

Given constraints: The end-to-end solution must provide secure data movement, high availability, and integration with Apache NiFi or ETL pipelines.

Proposed architecture:
Source systems → secure NiFi/ETL ingestion → HDFS landing layer → transformation/processing → analytics/output layer, with YARN managing cluster resources.

1. Secure data movement:
• Authenticate data producers and services.
• Use encrypted communication for data transferred between systems.
• Apply authorization and least-privilege access to Hadoop datasets.
• Validate and, when required, mask or filter sensitive records during ingestion.
• Maintain audit information so data movement and access can be traced.

2. Apache NiFi/ETL integration: NiFi can receive data from multiple sources, route it according to attributes, transform records and deliver data into Hadoop storage. ETL jobs can perform larger-scale transformations after raw data has been stored. Processing state and unique identifiers should be maintained to reduce duplicate ingestion.

3. HDFS high availability:
• Store large datasets in HDFS with appropriate block replication.
• Place replicas across different DataNodes/failure domains.
• Use NameNode high availability to reduce metadata-service downtime.
• Monitor DataNodes and under-replicated blocks and recover failed replicas.

4. YARN resource management: Use YARN to allocate CPU and memory to processing applications. Separate workloads into queues when multiple business applications share the cluster. This prevents one workload from consuming all available resources.

5. Processing and analytics: MapReduce or other Hadoop-compatible processing frameworks can process the stored data. Raw, transformed and curated data should be logically separated so that the organization can preserve source data and reproduce transformations when necessary.

6. Reliability and recovery: Combine HDFS replication, service redundancy, monitoring, verified backups and recovery procedures. Test failure scenarios such as DataNode loss, ingestion interruption and application failure.

Justification: Secure transport and access controls satisfy the security requirement; HDFS replication and NameNode high availability address availability; NiFi/ETL provides controlled integration; and YARN coordinates compute resources. Together these components form an end-to-end Hadoop ecosystem that directly addresses all three stated constraints.

Conclusion: Each solution explicitly prioritizes the constraints in the question before selecting Hadoop components. This matches the assessment rubric, which emphasizes understanding constraints, robust system design, effective application of Hadoop concepts, technical justification, and clarity.